

УРОВЕНЬ 1

БЛОК : ПРЕДСТАВЛЕНИЯ

1)

```
CREATE VIEW active_employees_info AS
SELECT
    e.id,
    e.first_name || ' ' || e.last_name AS full_name,
    d.name AS department_name,
    EXTRACT(YEAR FROM AGE(CURRENT_DATE, e.hire_date)) AS
years_of_service
FROM employees e
JOIN departments d ON e.department_id = d.id
WHERE e.is_active = true AND e.department_id IS NOT NULL;
```

2)

```
CREATE VIEW monthly_sales_summary AS
SELECT
    TO_CHAR(s.sale_date, 'YYYY-MM') AS year_month,
    p.category,
    SUM(s.quantity) AS total_quantity,
    SUM(s.quantity * p.price) AS total_revenue
FROM sales s
JOIN products p ON s.product_id = p.id
WHERE s.sale_date >= DATE_TRUNC('year', CURRENT_DATE - INTERVAL
'1 year')
GROUP BY TO_CHAR(s.sale_date, 'YYYY-MM'), p.category;
```

3)

```
CREATE VIEW available_books AS
SELECT
    b.id,
    b.title,
    a.name AS author_name
FROM books b
JOIN authors a ON b.author_id = a.id
WHERE NOT EXISTS (
    SELECT 1
    FROM book_loans bl
    WHERE bl.book_id = b.id AND bl.return_date IS NULL
);
```

БЛОК: ХРАНИМЫЕ ФУНКЦИИ

1)

```
CREATE OR REPLACE FUNCTION calculate_bonus(employee_id INT,
bonus_year INT)
RETURNS DECIMAL(10,2) AS $$
DECLARE
    emp_salary DECIMAL(10,2);
    emp_hire_date DATE;
    emp_active BOOLEAN;
    years_of_service INT;
    bonus_amount DECIMAL(10,2);
BEGIN
    SELECT salary, hire_date, is_active
    INTO emp_salary, emp_hire_date, emp_active
    FROM employees
    WHERE id = employee_id;

    IF NOT emp_active THEN
        RETURN 0;
    END IF;

    years_of_service := EXTRACT(YEAR FROM AGE(
        MAKE_DATE(bonus_year, 12, 31),
        emp_hire_date
    ));

    IF years_of_service > 5 THEN
        bonus_amount := emp_salary * 12 * 0.10;
    ELSE
        bonus_amount := emp_salary * 12 * 0.05;
    END IF;

    RETURN bonus_amount;
END;
$$ LANGUAGE plpgsql;
```

2)

```
CREATE OR REPLACE FUNCTION add_product_with_movement(
    p_sku VARCHAR,
    p_name VARCHAR,
    p_initial_stock INT
)
RETURNS INT AS $$
DECLARE
    new_product_id INT;
BEGIN
    INSERT INTO products (sku, name, current_stock)
    VALUES (p_sku, p_name, p_initial_stock)
```

```

        RETURNING id INTO new_product_id;

        INSERT INTO stock_movements (product_id, change_amount,
movement_type)
        VALUES (new_product_id, p_initial_stock, 'ADJUSTMENT');

        RETURN new_product_id;
END;
$$ LANGUAGE plpgsql;

```

3)

```

CREATE OR REPLACE FUNCTION get_total_estimated_time(task_id
INT)
RETURNS INT AS $$
DECLARE
    total_time INT;
BEGIN
    WITH RECURSIVE task_hierarchy AS (
        SELECT id, estimated_hours
        FROM task
        WHERE id = task_id

        UNION ALL

        SELECT t.id, t.estimated_hours
        FROM task t
        JOIN task_hierarchy th ON t.parent_task_id = th.id
    )
    SELECT COALESCE(SUM(estimated_hours), 0)
    INTO total_time
    FROM task_hierarchy;

    RETURN total_time;
END;
$$ LANGUAGE plpgsql;

```

4)

```

CREATE OR REPLACE FUNCTION get_employee_stats(
    department_id INT,
    OUT max_salary DECIMAL,
    OUT avg_salary DECIMAL,
    OUT emp_count INT
)
AS $$
BEGIN
    SELECT
        MAX(salary),
        AVG(salary),
        COUNT(*)
    INTO max_salary, avg_salary, emp_count

```

```
        FROM employees
        WHERE employees.department_id =
get_employee_stats.department_id;
END;
$$ LANGUAGE plpgsql;
```

БЛОК: ТРИГГЕРЫ

1)

```
CREATE OR REPLACE FUNCTION log_salary_change_func()
RETURNS TRIGGER AS $$
BEGIN
    IF OLD.salary IS DISTINCT FROM NEW.salary THEN
        INSERT INTO salary_audit_log (employee_id, old_salary,
new_salary)
            VALUES (NEW.id, OLD.salary, NEW.salary);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER log_salary_change
AFTER UPDATE ON employees
FOR EACH ROW
EXECUTE FUNCTION log_salary_change_func();
```

2)

```
CREATE OR REPLACE FUNCTION update_like_count_func()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        UPDATE posts
        SET like_count = like_count + 1
        WHERE id = NEW.post_id;
    ELSIF TG_OP = 'DELETE' THEN
        UPDATE posts
        SET like_count = like_count - 1
        WHERE id = OLD.post_id;
    END IF;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER update_like_count
AFTER INSERT OR DELETE ON likes
FOR EACH ROW
EXECUTE FUNCTION update_like_count_func();
```

3)

```

CREATE OR REPLACE FUNCTION check_milestone_dates_func()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.completed_date IS NOT NULL AND NEW.due_date IS NOT
NULL THEN
        IF NEW.completed_date < NEW.due_date THEN
            RAISE EXCEPTION 'Completed date cannot be earlier
than due date';
        END IF;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER check_milestone_dates
BEFORE INSERT OR UPDATE ON project_milestones
FOR EACH ROW
EXECUTE FUNCTION check_milestone_dates_func();

```

4)

```

CREATE OR REPLACE FUNCTION check_stock_availability()
RETURNS TRIGGER AS $$
DECLARE
    available_stock INT;
BEGIN
    SELECT stock INTO available_stock
    FROM products
    WHERE id = NEW.product_id;

    IF NEW.quantity > available_stock THEN
        RAISE EXCEPTION 'Cannot sell % units. Only % units
available in stock',
            NEW.quantity, available_stock;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER check_sales_stock
BEFORE INSERT OR UPDATE ON sales
FOR EACH ROW
EXECUTE FUNCTION check_stock_availability();

```

5)

```

CREATE VIEW employee_department_view AS
SELECT
    e.id AS employee_id,
    e.name AS employee_name,
    d.name AS department_name

```

```

FROM employees e
LEFT JOIN departments d ON e.department_id = d.id;

CREATE OR REPLACE FUNCTION instead_of_insert_employee_func()
RETURNS TRIGGER AS $$
DECLARE
    dept_id INT;
BEGIN
    SELECT id INTO dept_id
    FROM departments
    WHERE name = NEW.department_name;

    IF dept_id IS NULL THEN
        INSERT INTO departments (name)
        VALUES (NEW.department_name)
        RETURNING id INTO dept_id;
    END IF;

    INSERT INTO employees (name, department_id)
    VALUES (NEW.employee_name, dept_id);

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER instead_of_insert_employee
INSTEAD OF INSERT ON employee_department_view
FOR EACH ROW
EXECUTE FUNCTION instead_of_insert_employee_func();

```

БЛОК: КУРСОРЫ

1)

```

DO $$
DECLARE
    emp_cursor CURSOR FOR
        SELECT id, salary
        FROM employees
        WHERE hire_date < CURRENT_DATE - INTERVAL '10 years';
    emp_record RECORD;
    new_salary DECIMAL(10,2);
    salary_increase DECIMAL(10,2);
    counter INT := 0;
BEGIN
    OPEN emp_cursor;

    LOOP
        FETCH emp_cursor INTO emp_record;
        EXIT WHEN NOT FOUND;

        salary_increase := emp_record.salary * 0.05;
    
```

```

        IF salary_increase > 5000 THEN
            salary_increase := 5000;
        END IF;

        new_salary := emp_record.salary + salary_increase;

        UPDATE employees
        SET salary = new_salary
        WHERE id = emp_record.id;

        RAISE NOTICE 'Employee ID: %, New Salary: %',
emp_record.id, new_salary;

        counter := counter + 1;

        IF counter % 10 = 0 THEN
            COMMIT;
        END IF;
    END LOOP;

    CLOSE emp_cursor;
END;
$$;

```

2)

```

CREATE OR REPLACE FUNCTION get_product_sales_cursor(p_category
VARCHAR)
RETURNS REFCURSOR AS $$
DECLARE
    sales_cursor REFCURSOR := 'sales_cursor';
BEGIN
    OPEN sales_cursor FOR
        SELECT s.sale_date, s.quantity, p.name AS product_name
        FROM sales s
        JOIN products p ON s.product_id = p.id
        WHERE p.category = p_category
            AND s.sale_date >= DATE_TRUNC('month', CURRENT_DATE -
INTERVAL '1 month')
            AND s.sale_date < DATE_TRUNC('month', CURRENT_DATE);

    RETURN sales_cursor;
END;
$$ LANGUAGE plpgsql;

```

3)

```

CREATE OR REPLACE PROCEDURE
generate_monthly_report(report_month DATE)
LANGUAGE plpgsql AS $$
DECLARE
    book_cursor CURSOR FOR

```

```

        SELECT b.id, b.title, a.name AS author_name
        FROM books b
        JOIN authors a ON b.author_id = a.id;
    book_record RECORD;
    loan_count INT;
BEGIN
    OPEN book_cursor;

    LOOP
        FETCH book_cursor INTO book_record;
        EXIT WHEN NOT FOUND;

        SELECT COUNT(*)
        INTO loan_count
        FROM book_loans
        WHERE book_id = book_record.id
            AND DATE_TRUNC('month', loan_date) =
DATE_TRUNC('month', report_month);

        RAISE NOTICE 'Book: %, Author: %, Loans: %',
            book_record.title, book_record.author_name,
loan_count;
    END LOOP;

    CLOSE book_cursor;
END;
$$;

```

УРОБЕНЬ 2

1)

```

CREATE OR REPLACE FUNCTION get_department_employees(dept_id
INT)
RETURNS TABLE(employee_id INT, full_name TEXT, salary DECIMAL)
AS $$
BEGIN
    RETURN QUERY
    SELECT
        e.id,
        e.first_name || ' ' || e.last_name,
        e.salary
    FROM employees e
    WHERE e.department_id = dept_id
    ORDER BY e.salary DESC;
END;
$$ LANGUAGE plpgsql;

```

2)

```

CREATE OR REPLACE FUNCTION increment_like_counter()

```

```

RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        UPDATE posts
        SET like_count = like_count + 1
        WHERE id = NEW.post_id;
    ELSIF TG_OP = 'DELETE' THEN
        UPDATE posts
        SET like_count = like_count - 1
        WHERE id = OLD.post_id;
    END IF;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS update_like_count ON likes;

CREATE TRIGGER update_like_count
AFTER INSERT OR DELETE ON likes
FOR EACH ROW
EXECUTE FUNCTION increment_like_counter();

```

3)

```

CREATE MATERIALIZED VIEW mv_top_products AS
SELECT
    p.id,
    p.name,
    SUM(s.quantity) AS total_sold
FROM products p
JOIN sales s ON p.id = s.product_id
WHERE EXTRACT(YEAR FROM s.sale_date) = EXTRACT(YEAR FROM
CURRENT_DATE)
GROUP BY p.id, p.name
ORDER BY total_sold DESC
LIMIT 10;

CREATE OR REPLACE FUNCTION refresh_mv_top_products()
RETURNS VOID AS $$
BEGIN
    REFRESH MATERIALIZED VIEW mv_top_products;
END;
$$ LANGUAGE plpgsql;

CREATE OR REPLACE FUNCTION trigger_refresh_mv()
RETURNS TRIGGER AS $$
BEGIN
    PERFORM refresh_mv_top_products();
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

```

```
CREATE TRIGGER refresh_top_products_trigger
AFTER INSERT ON sales
FOR EACH STATEMENT
EXECUTE FUNCTION trigger_refresh_mv();
```

4)

```
CREATE TABLE IF NOT EXISTS daily_balances (
    id SERIAL PRIMARY KEY,
    account_id INT,
    balance_date DATE,
    end_of_day_balance DECIMAL(12,2)
);

CREATE OR REPLACE FUNCTION calculate_daily_balance(for_date
DATE)
RETURNS VOID AS $$
DECLARE
    trans_cursor CURSOR FOR
        SELECT account_id, amount, type
        FROM financial_transactions
        WHERE transaction_date = for_date
        ORDER BY account_id, id;
    trans_record RECORD;
    current_account INT := NULL;
    account_balance DECIMAL(12,2) := 0;
BEGIN
    OPEN trans_cursor;

    LOOP
        FETCH trans_cursor INTO trans_record;
        EXIT WHEN NOT FOUND;

        IF current_account IS NULL OR current_account !=
trans_record.account_id THEN
            IF current_account IS NOT NULL THEN
                INSERT INTO daily_balances (account_id,
balance_date, end_of_day_balance)
                VALUES (current_account, for_date,
account_balance);
            END IF;

            current_account := trans_record.account_id;
            account_balance := 0;
        END IF;

        IF trans_record.type = 'CREDIT' THEN
            account_balance := account_balance +
trans_record.amount;
        ELSIF trans_record.type = 'DEBIT' THEN
            account_balance := account_balance -
trans_record.amount;
```

```

        END IF;
    END LOOP;

    IF current_account IS NOT NULL THEN
        INSERT INTO daily_balances (account_id, balance_date,
end_of_day_balance)
        VALUES (current_account, for_date, account_balance);
    END IF;

    CLOSE trans_cursor;

    COMMIT;
END;
$$ LANGUAGE plpgsql;

```

5)

```

CREATE OR REPLACE FUNCTION audit_data_changes_func()
RETURNS TRIGGER AS $$
DECLARE
    old_data_json JSONB;
    new_data_json JSONB;
BEGIN
    IF TG_OP = 'DELETE' THEN
        old_data_json := row_to_json(OLD)::JSONB;
        new_data_json := NULL;

        INSERT INTO audit_trail (table_name, record_id,
old_data, new_data, operation)
        VALUES (TG_TABLE_NAME, OLD.id, old_data_json,
new_data_json, TG_OP);

        RETURN OLD;
    ELSIF TG_OP = 'UPDATE' THEN
        old_data_json := row_to_json(OLD)::JSONB;
        new_data_json := row_to_json(NEW)::JSONB;

        INSERT INTO audit_trail (table_name, record_id,
old_data, new_data, operation)
        VALUES (TG_TABLE_NAME, NEW.id, old_data_json,
new_data_json, TG_OP);

        RETURN NEW;
    ELSIF TG_OP = 'INSERT' THEN
        old_data_json := NULL;
        new_data_json := row_to_json(NEW)::JSONB;

        INSERT INTO audit_trail (table_name, record_id,
old_data, new_data, operation)
        VALUES (TG_TABLE_NAME, NEW.id, old_data_json,
new_data_json, TG_OP);

```

```
        RETURN NEW;  
    END IF;  
END;  
$$ LANGUAGE plpgsql;  
  
CREATE TRIGGER audit_data_changes  
AFTER INSERT OR UPDATE OR DELETE ON important_data  
FOR EACH ROW  
EXECUTE FUNCTION audit_data_changes_func();
```